

Libreto del taller

Sistemas Multi-Agente y Function Calling · Hackathon FISC AI AGENTS · UTP, salon 3-303

Como se lee esto

- El **texto negro grande es literal**: es lo que dices en voz alta. Lo que esta en **negrita** dentro de una frase, acentualo.
- Lo *gris y en cursiva* NO se dice: es lo que haces, o la respuesta que estas buscando.
- Lo **azul** son las preguntas que haces al salon. Despues de cada una, **callate** y espera.
- La **franja verde** es cuando ellos trabajan y tu no hablas. La mayor parte del taller no es tuya.
- La barrita gris corta es una pausa. Respira. No la llenes.

De donde salen los 70 minutos. Hablando: 1968 palabras, unos **14 min** a ritmo de presentacion. Ellos trabajando: **27 min** en las franjas verdes. Queda **29 min**, y no son relleno: son las corridas, la espera del modelo, las preguntas sueltas y algun 429. Si te oyes hablando mucho mas de 14 minutos, le estas quitando el tiempo a lo unico que se llevan puesto.

Apertura

Buenas. En el taller pasado vieron agentes, herramientas y memoria. Ya saben registrar una funcion con arroba-agente-punto-tool-plain, y ya vieron un agente decidir cual herramienta usar. Hoy vamos a lo que sigue: un agente que manda a otros agentes.

Y les digo de una lo que se llevan, porque esto no es teoria. En setenta minutos van a tener corriendo, en su maquina, un orquestador con dos subagentes. Y el ultimo ejercicio es el esqueleto del MVP que presentan el sabado. No es un ejemplo de juguete que despues botan. Es su punto de partida.

Antes de escribir una linea, la frase del día. Si se les olvida toda la sintaxis de hoy, esta no: **un subagente es un agente normal, llamado desde dentro de una herramienta de otro agente.**

Eso es todo. No hay una clase especial. No hay framework de orquestacion. No hay nada magico. Es una funcion que por dentro llama a otro agente. Cuando lo vean en el ejercicio tres les va a parecer poca cosa — y esta bien que les parezca poca cosa. Lo dificil no es escribirlo. Lo dificil es saber cuando vale la pena.

¿A quien le ha pasado que un agente se queda dando vueltas y le quema la cuota? Levanten la mano.

Levanta la tuya primero. Casi todos la levantan. Si no levanta nadie: "les va a pasar el sabado".

Eso tambien lo arreglamos hoy. Y no con un truco: con dos numeros que se ponen antes de la primera corrida.

Ultima cosa antes de arrancar. Hay dos modos de trabajo. MODO igual test no usa red ni llave: sirve para comprobar que su codigo corre. MODO igual gemini da respuestas de verdad y necesita la llave. La regla es: **armen en test, prueben en gemini.** Porque la capa gratis de Google da entre cinco y quince peticiones por minuto, y si prueban cada cambio contra el modelo real, a mitad del taller se quedan sin cuota.

Y cada quien necesita SU llave. No la compartan. Una llave entre treinta personas se muere en el primer ejercicio, y ahí el problema deja de ser de Google y pasa a ser mío.

4:00 – 20:00

Ejercicio 1 · la descripción es el prompt

Ejercicio uno. Abran ejercicios, cero-uno, herramientas punto py, y corranlo. Sin leerlo. Primero corre, después lee.

[Proyecta tu propia corrida mientras ellos corren la suya.]

Lo que acaban de ver es un agente con una herramienta que se llama buscar-texto. Y su docstring dice, completo: "Busca cosas." Dos palabras.

Fijense en lo que paso. No dio error. No se cayó. Hizo una llamada equivocada con total confianza. Y eso es lo peligroso, porque un error lo ves; una respuesta segura pero mal fundamentada te la crees.

Aquí está la idea del ejercicio, y es la que más les va a servir el sábado: **el modelo no ve su código**. No lee el cuerpo de la función. Ve tres cosas, nada más. El nombre de la función. Los tipos de los parámetros. Y el docstring. Esa es toda la interfaz.

O sea que su docstring no es documentación. Es prompt. Y si escriben un prompt de dos palabras, van a tener el comportamiento que merece un prompt de dos palabras.

Ahora arreglenlo. Un docstring que sirve dice tres cosas. Que devuelve, y en qué formato. Que NO hace, o que no acepta. Y que significa el resultado vacío.

La tercera es la que nadie escribe y es la que más cuesta. Si "Sin coincidencias" significa "no está", y no significa "fallo", díganlo. Si no lo dicen, el modelo va a asumir que fallo y va a reintentar en bucle. Y ahí se les va la cuota.

ELLOS TRABAJAN — tu te callas. 6 min. Arreglan el docstring y escriben su propia herramienta. Circula. Si alguien acaba temprano, pídele que lea su docstring en voz alta.

Una ultima cosa de este ejercicio, y haganla: pidanle un archivo que no existe.

El agente leyo el error y reintento. No se murio. Y eso no es suerte: es porque las funciones en taller, herramientas punto py, devuelven el error como texto en vez de lanzar excepcion. Si la funcion revienta, el agente se muere. Si devuelve "ERROR: el archivo no existe", el agente lo lee y corrige. Cuando escriban sus herramientas el sabado, devuelvan el error como texto.

20:00 – 27:00

Ejercicio 2 · que hace run_sync por debajo

Ejercicio dos. Es el mas corto, y es el que mas les va a servir cuando algo falle el sabado a las dos de la manana.

[Proyecta el bucle.]

run-sync no es magia. Es este bucle. Armas una lista de mensajes con el objetivo. Le pasas la lista al modelo. Si el modelo no pide herramientas, terminaste. Si pide, ejecutas la herramienta, le pegas la salida a la lista, y vuelves a llamar al modelo con la lista completa.

De ahi salen dos cosas. La primera: **el modelo no tiene memoria**. Ninguna. En cada vuelta se le reenvia el historial entero. Esa lista de mensajes ES la memoria. No hay nada mas.

La segunda sale de la primera: el paso diez carga los nueve anteriores. Por eso el costo de una corrida crece mas rapido que sus pasos. No es lineal.

ELLOS TRABAJAN — tu te callas. 3 min. Corren y miran el historial impreso. Que cuenten los ModelResponse: cada uno es una llamada que pagaron.

Ahora bajen request-limit a dos, en taller config punto py, y corran otra vez. Lean el mensaje.

Eso que acaban de ver es UsageLimits haciendo su trabajo: corto el agente antes de que siguiera gastando. Ese tope es lo unico que separa un bug de una factura. Ponganlo antes de la primera corrida, no despues del susto.

Si su MVP necesita treinta pasos, ¿que le pasa al historial? ¿Y al costo?

Dejalos contestar. NO contestes tu. La respuesta es la puerta al ejercicio 3; si la das tu, el ejercicio 3 pierde el porque.

Exacto. Y esa es la razon por la que existe el ejercicio tres.

27:00 – 55:00

Ejercicio 3 · el bucle multi-agente

Ejercicio tres. Este es el del taller. Si hoy solo se llevan un archivo, que sea este.

[Proyecta el patron de cinco lineas y dejalo en pantalla.]

Miren el patron. Son cinco lineas. Es una herramienta del orquestador, decorada con arroba-orquestador-punto-tool. Y por dentro, en vez de hacer el trabajo, llama a otro agente y devuelve su salida. Nada mas.

Y aqui esta lo importante: **para el orquestador esto es una herramienta cualquiera**. No sabe que por dentro corrio otro agente con su propio bucle de diez pasos. Ve una funcion que le devolvio un texto.

Ahora, por que importa. Y no es porque suene organizado. Hay cuatro razones, y una es la principal.

La principal es **aislamiento de contexto**. El subagente puede quemar cuarenta mil tokens leyendo archivos, y devolverle doscientas palabras al padre. Esos cuarenta mil nunca entran al historial del orquestador.

Y acuerdense del ejercicio dos: cada paso reenvia todo lo anterior. Si el orquestador cargara todo lo que leyeron sus hijos, reventaria en tres pasos. Por eso el aislamiento no es orden. Es lo que hace que la cosa funcione.

Las otras tres, rapido. Dos: menos herramientas por decision — cada agente elige entre tres opciones, no entre quince, y se equivoca menos. Tres: modelo por tarea — lo mecanico puede correr en algo mas barato. Y cuatro: permisos.

La de permisos mirenla en el codigo, porque es la que mas les va a importar el sabado. El explorador no tiene la herramienta escribir-archivo. No le pedimos que no escriba. **No tiene con que.**

Un prompt que dice "no borres nada" es una recomendacion. Una funcion que no existe es una garantia.

[Abre taller/herramientas.py y ensena _ruta_segura(). Son seis lineas.]

Lo mismo con el sandbox. Ningun agente de este repo puede tocar un archivo fuera de la carpeta sandbox. Y no es porque el prompt lo pida: es porque hay una funcion que resuelve la ruta y la rechaza si se sale. Seis lineas. Eso es una garantia; el prompt era una sugerencia.

Ahora les toca a ustedes, y es la tarea grande del dia. Van a escribir el segundo subagente: el escritor.

Los TODO en el archivo les dicen que tiene que cumplir, no como se escribe. Eso es a proposito. El explorador que esta justo arriba es su ejemplo, y esta funcionando: copien la forma, no el texto.

Tres pasos. Uno: el Agent, con sus instrucciones. Dos: su unica herramienta, escribir-archivo — solo esa. Si de paso le dan leer-archivo, deja de ser un escritor y el aislamiento de permisos se les cae. Y tres: registrenlo como herramienta del orquestador, con tool, no con tool-plain, porque necesitan el contexto.

ELLOS TRABAJAN — tu te callas. 12 min. Es la tarea principal del taller. Si alguien lleva mas de cinco minutos trabado en un typo, que abra soluciones/ y siga con el grupo: no se pierde veinte minutos ahi.

Ahora cambien el OBJETIVO a algo que necesite los dos — averiguar el bug y escribir un resumen — corran, y abran sandbox, reporte punto md.

Y fíjense en una cosa del sandbox, porque no es casualidad: el bug está repartido. Notas punto txt dice el síntoma: el login falla cuando el correo tiene mayúsculas. Y docs, arquitectura punto md, dice la causa: el correo se guarda sin normalizar. No se puede contestar leyendo un solo archivo. Eso es lo que justifica tener un explorador.

Miren también el reporte del final. Dice cuantos tokens quemo cada subagente, y cuantos caracteres devolvio. Esa segunda columna es lo único que entro al historial del orquestador. Lo de la primera se quedó adentro del hijo, y nadie lo paga dos veces.

En MODO=test la relación sale AL REVES: TestModel repite la salida de las herramientas en vez de resumir. El programa lo avisa solo. Si quieres que se vea de verdad, proyecta tu corrida en MODO=gemini.

Una más. Quitar usage igual ctx punto usage de una delegación, y corran.

¿Que le paso al conteo? El gasto del hijo dejó de contar para el padre. Y eso suena inofensivo hasta que lo dicen al revés: si el gasto del hijo no cuenta, **el límite del padre no está limitando nada**. Tres subagentes sueltos son tres presupuestos sin techo.

Última tarea, y es la que menos se enseña. Van a romper algo que no da error.

En la herramienta explorar, antes de llamar al subagente, ignoren lo que les pidió el orquestador y mandenle un encargo vago: "Dime que hay en el proyecto." Corran con el objetivo de antes, y abran reporte punto md.

Veán lo que paso. El explorador no falló. Contesto bien — contesto perfectamente una pregunta que no era la que hacía falta. Y el orquestador escribió su reporte igual de convencido. Cero errores. Ningun cuatrocientos veintinueve. Y el resultado está mal.

Ese es el costo real de delegar. No son los tokens del salto. Es que **el padre le encarga al hijo algo peor de lo que cree**. Cuando su MVP falle el sábado y los dos agentes "funcionen" — miren el encargo primero.

Ejercicio 4 · el esqueleto de tu MVP

Ejercicio cuatro. Este archivo es suyo y se lo llevan.

Primero en papel. Cinco minutos, sin teclado. Y no copien el ejemplo de archivos del ejercicio tres: pongan lo que su equipo va a construir de verdad.

Cuatro preguntas. Una: ¿que hace su MVP, en UNA frase? Dos: que subagentes necesita, y por cada uno, que herramientas SI tiene y que herramientas NO tiene. Tres: cual de las cuatro razones justifica separarlo. Cuatro: cual es su limite de peticiones.

De las cuatro, la que importa es **que herramientas NO tiene**. Si no pueden contestar eso de un subagente, todavia no saben que estan construyendo. Saben que quieren uno, que no es lo mismo.

Y el filtro, que es lo que me van a agradecer el sabado: si un subagente no aísla contexto, no acota permisos, no reduce decisiones y no cambia de modelo, **ese subagente no deberia existir**. Haganlo una herramienta normal del orquestador.

Porque multi-agente no es gratis. Cada salto son peticiones, latencia, y una traduccion mas donde se pierde informacion — la que acabamos de ver en la ultima tarea.

ELLOS TRABAJAN — tu te callas. 6 min. Circula. No revises codigo: pregunta "que hace tu MVP en una frase" y "cuantos subagentes y por que". Si no contestan en veinte segundos, ahi esta el trabajo.

Cierre

Vamos a cerrar. Dos ideas. Si se les olvida todo lo demás, estas dos aguantan.

La primera: **un subagente es una herramienta**, y su descripción es un prompt. Todo lo del ejercicio uno aplica igual al ejercicio tres.

Y de ahí sale el diagnóstico más útil que les puedo dar hoy: si el orquestador no está usando a su subagente, casi siempre el problema no es el subagente. Es el docstring.

La segunda: **lo que garantiza algo es el código, no el prompt**. El explorador no escribe porque no tiene la herramienta. Ningún agente se sale del sandbox porque hay una función que rechaza la ruta.

Cuando le den herramientas de escritura a un agente — y el sábado se las van a dar — esa es la diferencia que más les va a importar.

Y una tercera, de regalo: pongan los límites antes de la primera corrida. No después del susto.

¿Que van a construir? Una frase por equipo.

Dos o tres equipos, no más. Cierra con sus palabras, no con un resumen tuyo.